Mathematical Modelling Project

Adaptive Debt Collection Routing under Payment Uncertainty

Nguyen Thang Phuc
Student ID: 20252263M
Phuc.NT252263M@sis.hust.edu.vn

Dinh Thi Lan Huong
Student ID: 20261261M
Huong.DTL261261M@sis.hust.edu.vn

Supervisor: Viet Hung NGUYEN | viet_hung.nguyen@uca.fr

1 Introduction

Debt collection requires financial institutions to allocate limited staff time among customers located in different geographical areas. When in-person visits are necessary, collection officers must decide which customers to visit, in what order, and on which days. These decisions influence both travel requirements and the amount of money that may be recovered. Visiting every customer is often impractical because travel and service times must fit within daily working-hour limits. Furthermore, a visit does not guarantee payment, and an unsuccessful visit may require a subsequent attempt.

This project considers a multi-officer, multi-day debt collection routing problem in Hanoi. A team of collection officers operates from a common branch over a finite planning horizon. Each customer is associated with a target collection amount, an assumed payment probability, a service duration, and a geographical location. Each officer's daily route starts and ends at the branch, and total travel and service time must remain within the available working hours. Customers may be revisited subject to a maximum visit allowance, but a successfully collected amount must not be counted more than once.

The motivation is to allocate collection effort effectively under limited operational capacity and uncertain payment outcomes. Prioritizing nearby customers alone may overlook larger expected recoveries, whereas prioritizing large debts alone may lead to excessive travel or repeated unsuccessful visits. The problem therefore combines customer selection, visit scheduling, and route construction. The primary objective is to maximize expected recovered money. A secondary objective is to minimize total travel distance while preserving the primary objective value within a specified numerical tolerance.

The planning problem is formulated as a mixed-integer linear programming model and implemented in Julia using JuMP and the HiGHS solver. Heuristic approaches, including a genetic algorithm, tabu search, and adaptive large neighborhood search, are also investigated. The experimental study is designed to compare solution quality, computational time, and optimality gaps when valid solver bounds are available.

Two planning policies are considered. A static policy constructs a plan for the entire horizon before execution. A rolling-horizon policy updates customer states after each day and reoptimizes the remaining plan. Customers who have paid are removed from future visits, while unsuccessful visits reduce their remaining visit allowances. Simulated payment outcomes provide a common basis for comparing these policies.

The study uses a synthetic dataset informed by public demographic and credit-related data, together with geographical and road-network information. Customer accounts and payment outcomes are simulated and do not represent an actual bank's collection records. Payment probabilities are modelling assumptions rather than estimates validated against historical collection outcomes. The project therefore evaluates the behaviour and trade-offs of the proposed methods under controlled scenarios, rather than claiming verified improvements in real banking operations.

2 Mathematical Model

2.1 Problem Assumptions

The final implementation relies on the following assumptions.

- **Full-or-zero payment.** A successful visit to customer i collects the full target amount w_i; an unsuccessful visit collects nothing. Partial payments are not modelled.
- **Conditional payment probability.** p_{it} is the probability that customer i pays when visited on day t, given that the customer has not paid before. Payment outcomes of different visits are conditionally independent, and p_{it} decreases with t.
- **Maximum visits.** Customer i can be visited at most m_i times over the horizon. All experiments use $m_i = 2$.
- **At most one visit per customer per day**, made by exactly one officer. Different visits to the same customer may be made by different officers.
- **Officer eligibility.** Every officer may visit every customer; the officers are identical except for their (equal) daily time budgets.
- **Daily routes.** Every active officer leaves the branch once per day and returns to it before the working-time budget is exhausted. The budget covers travel time and service time.
- **After a successful payment** the debt is settled: the amount is counted once, and the customer receives no further visits. In the static plan this is handled during execution (Section 2.6); in the planning model it is captured by the expected value of a visit schedule.

2.2 Input Data and Notation

Table 1 lists the notation. Money is expressed in million VND, time in hours and distance in kilometers. Travel times and distances come from shortest (fastest) paths on a directed motorbike road network, so in general $d_{ij} \neq d_{ji}$ and $\tau_{ij} \neq \tau_{ji}$.

Table 1: Main notation.

Symbol	Description
N	Set of customers
K	Set of collection officers
T	Set of planning days
$V = N \cup \{0\}$	Routing nodes, where 0 is the branch
$A = \{(i,j) \in V \times V : i \neq j\}$	Arcs of the complete directed routing graph
$n = \lvert N \rvert$	Number of customers
w_i	Target collection amount of customer i (million VND)
p_{it}	Payment probability for customer i on day t, given no earlier payment
d_{ij}	Travel distance from node i to node j (km)
τ_{ij}	Travel time from node i to node j (hours)
s_i	Service duration at customer i (hours)
B_{kt}	Working-time budget of officer k on day t (hours; 8 h in all instances)
m_i	Maximum number of visits to customer i
$\mathcal{S}_i$	Feasible visit schedules of customer i: $\{S \subseteq T : 1 \leq \lvert S \rvert \leq m_i\}$
a_{tS}	1 if $t \in S$, 0 otherwise
v_{iS}	Precomputed expected collection of customer i under schedule S (million VND)

2.3 Decision Variables

All variables are indexed by $i, j \in V$, $k \in K$, $t \in T$ and $S \in \mathcal{S}_i$.

- $x_{ijkt} \in \{0,1\}$, $(i,j) \in A$: equals 1 if officer k travels directly from node i to node j on day t (arc variables).
- $y_{ikt} \in \{0,1\}$, $i \in N$: equals 1 if officer k visits customer i on day t (visit assignment).
- $z_{kt} \in \{0,1\}$: equals 1 if officer k performs a route on day t (route activation).
- $\lambda_{iS} \in \{0,1\}$, $i \in N$: equals 1 if customer i is visited exactly on the set of days S (schedule selection).
- $u_{ikt} \geq 0$, $i \in N$: position of customer i in the route of officer k on day t; used only for subtour elimination (Miller–Tucker–Zemlin variables [5]).

2.4 Objective Function

With full-or-zero payment and conditionally independent visits, the probability that customer i pays at least once under schedule S is $1 - \prod_{t \in S}(1 - p_{it})$, so its expected collection is

$$v_{iS} = w_i \Big[1 - \prod_{t \in S} (1 - p_{it})\Big]. \tag{1}$$

This value is not additive over the visits: a second visit only adds value if the first one fails. Expressing it directly in terms of y_{ikt} would require products of binary variables. Instead, v_{iS} is computed before solving for every schedule in $\mathcal{S}_i$, and the model selects one schedule per customer. The objective is therefore linear:

$$\max \; Z = \sum_{i \in N} \sum_{S \in \mathcal{S}_i} v_{iS} \, \lambda_{iS}. \tag{2}$$

With $H = 5$ days and $m_i = 2$, each customer has $|\mathcal{S}_i| = \binom{5}{1} + \binom{5}{2} = 15$ schedules.

Secondary objective. Let $Z^\star$ be the value obtained in the first solve. A second solve keeps the expected collection within a relative tolerance $\varepsilon = 10^{-6}$ and minimizes the total distance:

$$\sum_{i \in N} \sum_{S \in \mathcal{S}_i} v_{iS} \lambda_{iS} \;\geq\; Z^\star - \varepsilon \max(1, |Z^\star|), \tag{3}$$

$$\min \sum_{t \in T} \sum_{k \in K} \sum_{(i,j) \in A} d_{ij} \, x_{ijkt}. \tag{4}$$

The second solve starts from the first solution and has its own time limit. It was *not* enabled in the algorithm-comparison experiments (Section 4), where only (2) is optimized. In the heuristics, distance is always used as a lexicographic tie-breaker between solutions with equal expected value.

2.5 Constraints

$$\sum_{S \in \mathcal{S}_i} \lambda_{iS} \le 1 \qquad \forall i \in N, \tag{5}$$

$$\sum_{k \in K} y_{ikt} = \sum_{S \in \mathcal{S}_i} a_{tS} \lambda_{iS} \qquad \forall i \in N,\ t \in T, \tag{6}$$

$$\sum_{j \in N} x_{0jkt} = z_{kt}, \qquad \sum_{i \in N} x_{i0kt} = z_{kt} \qquad \forall k \in K,\ t \in T, \tag{7}$$

$$\sum_{j \in V \setminus \{i\}} x_{ijkt} = y_{ikt}, \qquad \sum_{j \in V \setminus \{i\}} x_{jikt} = y_{ikt} \qquad \forall i \in N,\ k \in K,\ t \in T, \tag{8}$$

$$\sum_{(i,j) \in A} \tau_{ij} x_{ijkt} + \sum_{i \in N} s_i y_{ikt} \le B_{kt} z_{kt} \qquad \forall k \in K,\ t \in T, \tag{9}$$

$$u_{ikt} - u_{jkt} + n\, x_{ijkt} \le n - 1 + n(1 - y_{jkt}) \qquad \forall i, j \in N,\ i \ne j,\ k \in K,\ t \in T, \tag{10}$$

$$y_{ikt} \le u_{ikt} \le n\, y_{ikt} \qquad \forall i \in N,\ k \in K,\ t \in T, \tag{11}$$

$$x_{ijkt},\ y_{ikt},\ z_{kt},\ \lambda_{iS} \in \{0, 1\}, \quad u_{ikt} \ge 0. \tag{12}$$

- **Schedule selection** (5): each customer gets at most one schedule. The inequality allows a customer to be skipped. The visit limit m_i is enforced by the definition of $\mathcal{S}_i$.
- **Schedule–visit consistency** (6): if the chosen schedule contains day t, exactly one officer visits customer i on that day; otherwise nobody does. This also rules out two visits to the same customer on the same day.
- **Branch departure and return** (7): an active officer leaves and re-enters the branch exactly once per day; an inactive officer uses no arc.
- **Flow conservation** (8): a visited customer has exactly one incoming and one outgoing arc on the route of the officer who visits them, and none on other routes.
- **Working-time limit** (9): travel plus service time of officer k on day t does not exceed B_{kt}; if $z_{kt} = 0$ the officer does no work that day.
- **Subtour elimination** (10)–(11): if $x_{ijkt} = 1$ and j is visited, then $u_{jkt} \ge u_{ikt} + 1$, so positions strictly increase along a route and no cycle disconnected from the branch can exist. The term $n(1 - y_{jkt})$ deactivates the constraint when j is not on the route, and (11) forces $u_{ikt} = 0$ for unvisited customers.
- **Domains** (12).

Symmetry breaking. Officers with the same budget are interchangeable: relabelling two officers gives a different solution with the same value, and the branch-and-bound tree explores such duplicates. For consecutive officers with $B_{kt} = B_{k+1,t}$ the code adds

$$z_{kt} \ge z_{k+1,t} \qquad k = 1, \ldots, |K| - 1,\ t \in T, \tag{13}$$

which removes duplicates without changing the optimal value. It was enabled in all algorithm-comparison experiments.

2.6 Rolling-Horizon Operation

The model above produces a *static* plan for the whole horizon. During execution, payment outcomes are revealed day by day. The state known at the start of day t is (P_t, a_t), where P_t is the set of customers who have already paid and a_{it} is the number of visits made to customer i before day t. The eligible customers are $A_t = \{i \in N : i \notin P_t,\ a_{it} < m_i\}$.

The rolling-horizon policy repeats the following steps for $t = 1, \ldots, H$:

1. Build the remaining problem: customers A_t, days $t, \ldots, H$, probabilities $p_{i,t..H}$, budgets $B_{k,t..H}$, remaining visit allowances $m_i - a_{it}$, and the travel sub-matrix of the branch and the customers

in A_t. The probabilities are not re-estimated: because p_{it} is already conditional on no earlier payment, it remains valid for every customer still in A_t.
2. Solve the same model (2)–(12) over the remaining horizon.
3. Execute only the routes of day t; the plan for later days is discarded.
4. Observe the simulated payments of the visited customers, add the payers to P_{t+1} and increase $a_{i,t+1}$ for every visited customer.

Two further policies serve as references. The *static* policy solves once at the start and executes the whole plan; a customer who has already paid is skipped, and the officer goes straight to the next unpaid customer on the route without inserting new customers. The *myopic* policy works like the rolling horizon but optimizes only day t each morning ($H = 1$ in the remaining problem), so constraint (6) no longer links it to later days.

All three policies produce feasible routes. The rolling-horizon procedure is *not* a fully adaptive stochastic model: each re-solve maximizes the expected collection as if the remaining plan were executed without further re-planning. It does not anticipate that future decisions will depend on future outcomes, and it is not an optimal policy of the underlying multi-stage stochastic problem. Solver bounds certify only the remaining problem at each re-solve, not the policy.

3 Solution Methods

3.1 MILP Solution with HiGHS

The model is built with JuMP [7] and solved by the branch-and-cut algorithm of HiGHS [8], with all of x, y, z, λ declared binary. HiGHS performs presolve, solves the LP relaxation at the root node, adds cutting planes, runs its primal heuristics and branches on fractional variables, while maintaining an incumbent and an upper bound on (2). The solver stops when the relative gap falls below 10^{-4} (`mip_rel_gap`) or when the time limit is reached: 30 s for the one-day problem and 150 s for the five-day problem (Section 4). The HiGHS random seed is set to the seed of each run. The reported status is `OPTIMAL` or `TIME_LIMIT`, and the bound and branch-and-bound node count are read from the solver after the main solve.

Warm start. In the warm-started variant, an ALNS solution obtained with the same seed is passed to HiGHS as a MIP start (values of x, y, z, u and of the matching λ_{iS}). The ALNS time is not included in the solver time. If HiGHS stops without a solution at least as good as the start, the code returns the ALNS solution and labels the run `TIME_LIMIT_HEURISTIC_FALLBACK`. Such a solution is reported as an ALNS solution, not as one found by HiGHS; this happened in 6 of the 70 warm-started five-day runs (1 on INS_S with 50 customers, 2 on REP1_S and 3 on REP2_S). The solver bound remains valid in all cases.

Shared heuristic framework. The three metaheuristics below use the same representation: an assignment matrix $G \in \{0, 1, \ldots, |K|\}^{n \times H}$, where $g_{it} = 0$ means that customer i is not visited on day t and $g_{it} = k$ means that officer k visits them. A cell holds one value, so a customer cannot be visited by two officers on the same day. For every officer–day pair, the route through the assigned customers is built by nearest-neighbor from the branch and improved by 2-opt, where every reversal is evaluated on the full (asymmetric) distance. A shared *repair* procedure restores feasibility: if a customer has more than m_i visits, only the m_i days with the highest p_{it} are kept; if a route exceeds B_{kt}, the visit with the smallest marginal loss $v_{iS_i} - v_{i,S_i \setminus \{t\}}$ is removed and the route rebuilt, until the route fits. Solutions are compared lexicographically: higher expected collection first, then shorter total distance. Because the order within a route is built by a rule rather than optimized, the heuristics may miss routes that the MILP can find, and they provide no optimality certificate.

3.2 Genetic Algorithm

- **Encoding:** one individual is one matrix G.
- **Initialization:** each cell is selected for a visit with probability 0.35 and assigned to a random officer; the individual is then repaired.
- **Selection:** tournament selection of size 3, based on the lexicographic comparison above.
- **Crossover:** uniform crossover; each cell of the child is copied from one of the two parents with equal probability.
- **Mutation:** each cell is replaced, with probability 0.03, by 0 or a random officer.
- **Repair and elitism:** every child is repaired; the 4 best individuals are copied unchanged into the next generation.
- **Parameters and stopping:** population 80, at most 300 generations, stopping early after 50 generations without improvement of the best solution.

3.3 Tabu Search

- **Initial solution:** candidate visits (i, t) are ranked by $w_i p_{it}/(s_i + \tau_{0i} + \tau_{i0})$ and inserted greedily while the solution stays feasible.
- **Neighborhood:** a move picks a random cell (i, t) and changes its value to "no visit" or to another officer, i.e. it adds, removes or reassigns one visit. Each iteration samples 40 such moves, repairs and evaluates them, and moves to the best admissible neighbor even if it is worse than the current solution. A neighbor whose repair undoes the proposed change is discarded.
- **Tabu memory:** returning a cell to its previous value is forbidden for 15 iterations.
- **Aspiration:** a tabu move is allowed if it yields a new best solution.
- **Stopping:** at most 300 iterations, or 50 consecutive iterations without improving the best solution.

3.4 Adaptive Large Neighborhood Search

ALNS [6] removes a large part of the plan and rebuilds it.

- **Destroy operators** (remove about 20% of the current visits): random removal; worst-value removal (lowest marginal expected value); related removal (a random customer and the customers closest to them); route removal (part of one officer–day route).
- **Repair operators:** greedy insertion, which inserts the feasible triple (i, t, k) with the largest gain in expected value, breaking ties by the smallest increase in distance; and regret-2 insertion, which favors customers who lose the most if their best insertion is not used. Customers not in the plan may be inserted. Insertion continues until no feasible insertion remains.
- **Operator selection and weights:** roulette-wheel selection by operator weight. Operators score 10 for a new global best, 5 for improving the current solution and 1 for an accepted solution. Every 25 iterations the weights are updated as $w \leftarrow 0.8\, w + 0.2\, \bar{\pi}$, where $\bar{\pi}$ is the mean score in the segment (floored at 0.1).
- **Acceptance:** simulated annealing on the score $Z/\sum_i w_i - 10^{-4} \times \text{km}$, with initial temperature 0.05 and cooling factor 0.995 per iteration; the best solution is still kept by the lexicographic comparison.
- **Stopping:** at most 300 iterations, or 50 consecutive iterations without improving the best solution.

4 Numerical Experiments

4.1 Dataset and Test Instances

The dataset [1] combines three kinds of inputs.

Observed public data. The road network, the 2025 ward boundaries and populations, building density and the branch location (a bank point of interest in central Hanoi) come from OpenStreetMap [2]. After keeping the largest strongly connected motorbike network, the graph has 57,242 nodes and 130,220 directed edges, 6.6% of them one-way. Demographic profiles (gender, age, marital status, education, occupation) are taken unchanged from NVIDIA Nemotron-Personas-Vietnam [3].

Synthetic inputs informed by public data. Customer locations are sampled in 27 wards with quotas proportional to ward population, at road nodes weighted by nearby building density. The shape of the credit-limit and card-utilization distributions is taken from the UCI Default of Credit Card Clients dataset [4] (customers at least two months late), and the shares of debt groups 2–5 follow published Vietnamese bank reports. Loan amounts, interest rates and terms of other products are drawn from assumed log-normal distributions. The target amount w_i is the overdue amount.

Modelling assumptions. The payment probabilities $p_{it} = h\, p_0(g)\,(1 - 0.3 f_i)\,(1 - \delta(g))^{t-1}$, capped at 0.95, depend only on the debt group g, the position f_i of the customer's days past due within the group, and the day t; $h = 0.5/1/1.5$ defines the *low*, *base* and *high* scenarios (THAP, CO_SO, CAO). Road speeds by road type, service times, the 480-minute working day and the planning period (five working days) are also assumed. The probabilities are not calibrated against real collection outcomes.

The model-layer graph is complete and directed: $|V| = n + 1$ nodes and $|A| = n(n + 1)$ arcs, where each arc carries the time and distance of the fastest road path, computed with Dijkstra's algorithm. The matrix is asymmetric for 95.8% of the node pairs. Figure 1 illustrates the two graph layers.

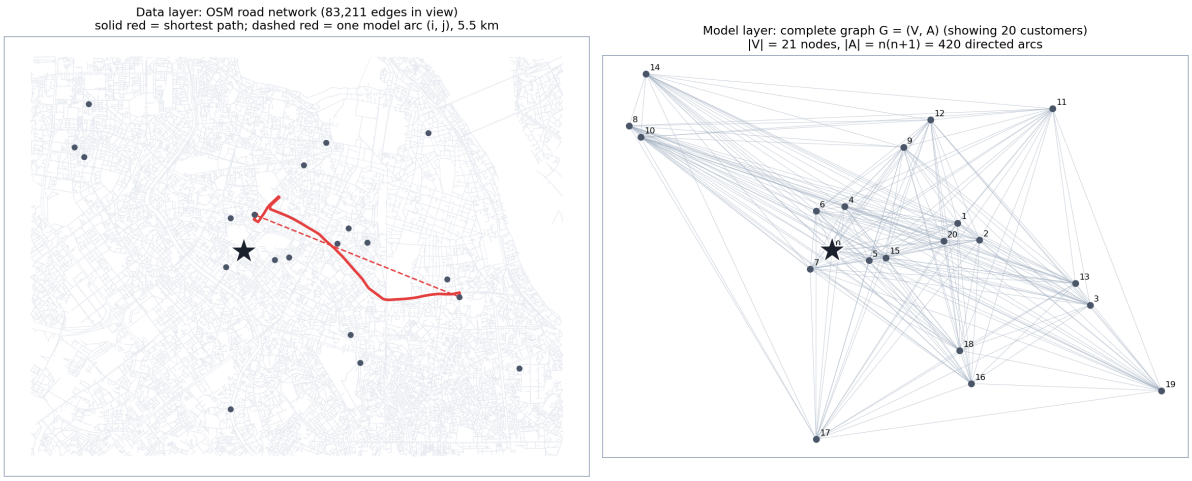


Figure 1: Left: the road network (data layer) and the fastest road path behind one model arc (i, j). Right: the complete directed graph of the model (model layer) for the first 20 customers of INS_XS.

Five customer sets were generated with different seeds: MAIN (500 customers) and REP1–REP4

(200 each). The instances are nested (INS_XS $\subset$ INS_S $\subset$ INS_M $\subset$ INS_L from MAIN, REPk_S $\subset$ REPk_M from REPk); Table 2 lists their default parameters. To isolate the effect of size, the experiments fix $|K| = 2$, $B_{kt} = 8\,\text{h}$, $m_i = 2$ and the base scenario, and use the first n customers of INS_XS ($n = 10, 20, 30$) or INS_S ($n = 50, 80$). REP1_S–REP4_S (one-day problem) and REP1_S–REP2_S (five-day problem) are added at $n = 80$ to check that the conclusions do not depend on one customer set. INS_M and INS_L were not used in the reported experiments.

Table 2: Test instance characteristics (dataset defaults; all have a 480-minute working day).

Instance	Customers	Officers	Days	Max. visits
INS_XS	30	2	5	3
INS_S	80	2	5	2
INS_M	200	3	5	2
INS_L	500	4	5	2
REPk_S, $k = 1..4$	80	2	5	2
REPk_M, $k = 1..4$	200	3	5	2

4.2 Computational Environment

All reported experiments were run on the laptop described in Table 3. The one-day and five-day experiments ran in parallel as two separate processes; since the HiGHS MIP solver runs almost serially, the two processes did not compete significantly for CPU.

Table 3: Computational environment.

Item	Configuration
Processor	Intel Core i7-1165G7, 2.80 GHz, 4 cores / 8 threads
Memory	8 GB
Operating system	Windows 10 Home 22H2 (build 19045)
Julia version	1.13.2 (single-threaded)
JuMP / HiGHS versions	JuMP 1.32.1, MathOptInterface 1.54.0; HiGHS.jl 1.26.0 (HiGHS_jll 1.15.1)
Solver threads	HiGHS default settings; the MIP search is essentially serial
Code commit	`a9c6bf0`; solver code identical to `05375a4`, at which the algorithm-comparison results were committed

4.3 Experimental Protocol

Algorithm comparison (repeated optimization runs). The study has two phases, mirroring the single-period and multi-period steps of the course's reference problem: *Phase 1* solves a one-day problem ($H = 1$, so each customer is visited at most once and no day linking occurs) and *Phase 2* the five-day problem ($H = 5$). Each configuration is solved in 10 independent runs $r = 1, \ldots, 10$ with seed $2026 + r$, used by every method of that run, and all reported values are averages over the 10 runs. The HiGHS time limit is proportional to the number of days, as in the reference problem: 30 s in Phase 1 and $150\,\text{s} = 5 \times 30\,\text{s}$ in Phase 2, the same for every instance of a phase. The metaheuristics are not given a time limit but the same iteration budget: at most 300 iterations or generations, stopping after 50 consecutive iterations without improvement (parameters in Section 3). In Phase 2, HiGHS is run twice per run: cold (no start) and warm-started from the ALNS solution of the same seed.

Timing. For HiGHS, the reported time is the wall-clock time of the `optimize!` call; reading the CSV files, building the instance and building the JuMP model are excluded, and so is the

ALNS run that provides the warm start. For the metaheuristics, the time is the wall-clock time of the solver call (construction, search and repair). Julia compiles each function on its first call, and this compilation time is included in the first run of the first instance (for example, Tabu took 5.8 s in run 1 on the 10-customer Phase 2 instance against a mean of 0.72 s). Total elapsed time per run, including model construction, was not recorded separately.

Policy comparison (Monte Carlo payment scenarios). These scenarios are distinct from the repeated optimization runs: the routing method is fixed (ALNS, seed 2026, 300 iterations) and the uncertainty comes from the payment outcomes. The instance is INS_S (80 customers, 2 officers, 5 days, $m_i = 2$), evaluated under the three probability scenarios with 20 payment scenarios $s = 1, \ldots, 20$ each. The dataset stores a uniform number $\xi_{it}^{(s)} \in [0, 1]$ for every customer and day; a visit to customer i on day t in scenario s succeeds if $\xi_{it}^{(s)} < p_{it}$, and the outcome is only drawn when the visit is actually made. The same numbers are used by all policies (common random numbers), so the paired differences are due to the policies alone. The algorithms never see $\xi^{(s)}$ when planning; it is used only to generate the outcome after the routes of a day are chosen. In the static policy, a planned visit to a customer who has already paid is skipped: the officer drives straight to the next unpaid customer on the route, and the freed time is not used. The static plan was solved once per probability scenario with all 300 ALNS iterations, whereas each daily re-solve of the rolling-horizon and myopic policies used the early-stopping rule (50 iterations without improvement).

4.4 Evaluation Metrics

- **Expected collection** Z (million VND): the value of objective (2) for the returned plan, i.e. the expectation *before* execution, not an amount that is certain to be collected.
- **Simulated collected money** (million VND): the sum of w_i over the successful visits in one payment scenario, averaged over the scenarios. Only this quantity is used to compare policies.
- **Distance** (km): total length of the executed routes in the policy comparison. Distances were not recorded in the algorithm-comparison runs, which optimize only Z.
- **Time** (s): as defined in the protocol above.
- **Optimality gap.** In each run, UB^* is the tightest (smallest) upper bound returned by the HiGHS calls of that run (cold and, in Phase 2, warm), and the gap of every method in that run is

$$\text{gap} = \frac{UB^* - Z}{Z} \times 100\%. \tag{14}$$

  The denominator is the solution value, as in the course's reference formula; values below 10^{-9} are reported as 0. A gap of 0 means the solution is proved optimal. For the heuristics the gap is conservative: when UB^* is loose, the true distance to the optimum may be smaller. A gap is reported only if both a feasible solution and a finite bound exist; otherwise it is left empty and never treated as 0. In all reported runs, every method returned a feasible solution and HiGHS returned a finite bound.
- **Branch-and-bound nodes:** the HiGHS node count (`MOI.NodeCount`) of the main solve; 0 or 1 means that the solver stopped at the root node. N/A for the heuristics.
- **Optimal runs:** the number of runs, out of 10, in which HiGHS reported status `OPTIMAL` (gap $\leq 10^{-4}$).

4.5 Algorithm Comparison

Table 4 reports the main instances; the full results for $n = 10$ and $n = 20$ and per-run data are in the repository [1]. For $n = 10$ and $n = 20$ the picture is the same as for $n = 30$: HiGHS is optimal in 10/10 runs in both phases and every metaheuristic reaches the optimum. Every method returned a feasible solution and HiGHS returned a finite bound in all 160 runs (90 in

Phase 1, 70 in Phase 2), so no gap had to be left undefined. Distances were not recorded in these runs, so the table has no distance column; it shows the number of runs proved optimal instead.

Table 4: Algorithm comparison: averages over ten runs. Z is expected collection in million VND with its standard deviation over the runs in parentheses; time is in seconds. "Warm" is HiGHS started from the ALNS solution.

Instance	Method	Z	Time	Gap (%)	B&B nodes	Optimal
Phase 1: one day, 30 s limit						
INS_XS, $n = 30$	HiGHS	139.18 (0.00)	1.76	0.00	1.0	10/10
	GA	139.18 (0.00)	2.75	0.00	N/A	N/A
	Tabu	139.18 (0.00)	1.25	0.00	N/A	N/A
	ALNS	139.18 (0.00)	1.05	0.00	N/A	N/A
INS_S, $n = 50$	HiGHS	68.51 (41.51)	30.63	270.88	3.5	0/10
	GA	200.90 (0.24)	6.76	0.64	N/A	N/A
	Tabu	200.83 (0.28)	2.28	0.67	N/A	N/A
	ALNS	200.94 (0.23)	7.14	0.62	N/A	N/A
INS_S, $n = 80$	HiGHS	38.44 (33.49)	31.98	877.24	0.0	0/10
	GA	274.32 (0.96)	7.71	2.40	N/A	N/A
	Tabu	273.43 (0.52)	2.21	2.74	N/A	N/A
	ALNS	274.81 (0.61)	13.91	2.22	N/A	N/A
Phase 2: five days, 150 s limit						
INS_XS, $n = 30$	HiGHS	246.80 (0.00)	19.62	0.00	56.5	10/10
	HiGHS warm	246.80 (0.00)	1.94	0.00	0.0	10/10
	GA	246.80 (0.00)	10.21	0.00	N/A	N/A
	Tabu	246.80 (0.00)	2.57	0.00	N/A	N/A
	ALNS	246.80 (0.00)	3.54	0.00	N/A	N/A
INS_S, $n = 50$	HiGHS	35.82 (16.61)	155.03	1,264.36	2.0	0/10
	HiGHS warm	361.90 (0.03)	154.74	0.08	1.8	0/10
	GA	361.59 (0.14)	18.34	0.16	N/A	N/A
	Tabu	361.56 (0.16)	4.98	0.17	N/A	N/A
	ALNS	361.90 (0.03)	9.91	0.08	N/A	N/A
INS_S, $n = 80$	HiGHS	6.99 (6.70)	156.76	16,594.40	0.0	0/10
	HiGHS warm	549.04 (0.22)	157.37	0.38	0.0	0/10
	GA	542.49 (2.13)	23.30	1.59	N/A	N/A
	Tabu	545.20 (0.58)	5.80	1.09	N/A	N/A
	ALNS	549.04 (0.22)	30.67	0.38	N/A	N/A

Figure 2 plots the mean gap and time against n for all five sizes. Table 5 repeats the 80-customer comparison on the independently generated customer sets.

Table 5: Mean gap (%) at $n = 80$ on independent customer sets, ten runs each. HiGHS (cold) was not optimal in any run.

Phase	Instance	HiGHS	HiGHS warm	GA	Tabu	ALNS
1	INS_S	877.24	—	2.40	2.74	2.22
	REP1_S	2,005.39	—	3.28	3.63	3.68
	REP2_S	482.99	—	3.31	3.71	3.70
	REP3_S	24,330.70	—	2.27	2.57	2.23
	REP4_S	5,872.45	—	2.70	2.73	2.70
2	INS_S	16,594.40	0.38	1.59	1.09	0.38
	REP1_S	41,049.20	0.49	2.56	1.18	0.49
	REP2_S	14,769.40	0.45	2.21	1.01	0.45

4.6 Static and Rolling-Horizon Comparison

The comparison uses INS_S with ALNS as the routing method and 20 common payment scenarios per probability scenario. Table 6 reports means of the *simulated* collection, not of the optimization objective; for reference, the expected value of the static plan was 287.98, 549.49 and 783.45 million VND in the low, base and high scenarios. Time is the total optimization time for one

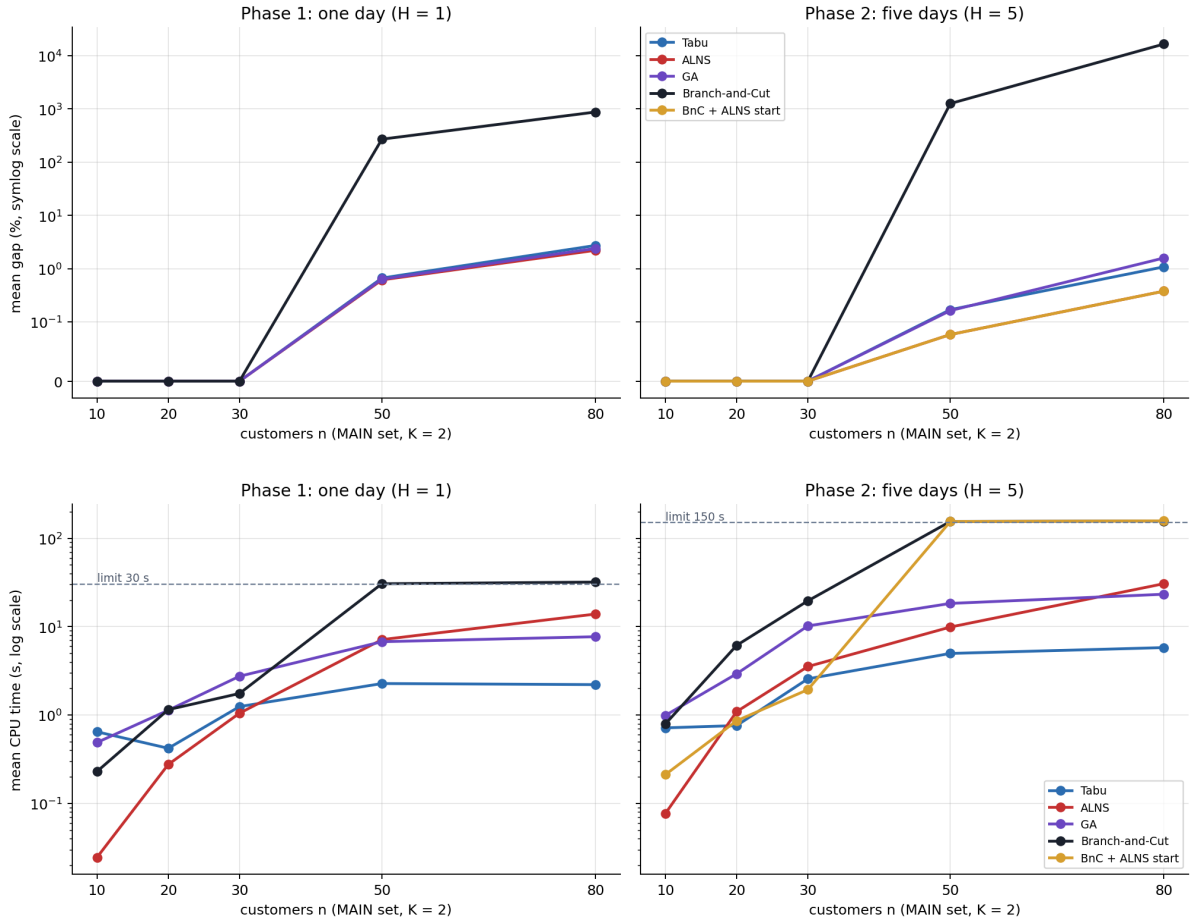


Figure 2: Mean optimality gap (top, symmetric log scale) and mean time (bottom, log scale; dashed line: HiGHS time limit) against the number of customers on the MAIN set, ten runs per point.

scenario: the single static solve, or the sum of the five daily re-solves. Table 7 gives the paired differences with 95% confidence intervals based on the t distribution with 19 degrees of freedom. All statistics are produced by the runner from the per-scenario results [1].

Table 6: Policy comparison on INS_S over 20 common payment scenarios (ALNS). Collected amounts are in million VND; standard deviations over the scenarios in parentheses.

Scenario	Policy	Mean collected	Mean km	Mean total time (s)
Low (THAP)	Static	289.62 (151.66)	325.4	118.2
	Rolling horizon	288.41 (155.85)	306.0	80.4
	Myopic	289.18 (161.48)	283.7	42.8
Base (CO_SO)	Static	514.38 (207.46)	316.0	106.9
	Rolling horizon	516.67 (203.61)	288.3	60.5
	Myopic	507.99 (209.02)	261.6	35.5
High (CAO)	Static	735.90 (189.66)	310.4	108.6
	Rolling horizon	740.83 (196.05)	285.3	58.1
	Myopic	733.40 (204.62)	259.6	27.5

Table 7: Paired differences against the static policy over the same 20 scenarios: mean difference, 95% confidence interval, and number of scenarios in which the policy is better.

		Collected (million VND)		Distance (km)	
Scenario	Policy	Mean [95% CI]	Better	Mean [95% CI]	Shorter
Low	Rolling	−1.21 [−8.46, 6.05]	10/20	−19.5 [−25.0, −13.9]	18/20
	Myopic	−0.44 [−10.99, 10.10]	11/20	−41.7 [−45.2, −38.2]	20/20
Base	Rolling	+2.28 [−4.28, 8.84]	12/20	−27.7 [−31.9, −23.5]	20/20
	Myopic	−6.40 [−21.09, 8.30]	9/20	−54.4 [−57.8, −50.9]	20/20
High	Rolling	+4.93 [−8.09, 17.95]	11/20	−25.0 [−30.4, −19.7]	20/20
	Myopic	−2.50 [−23.37, 18.37]	9/20	−50.8 [−54.8, −46.8]	20/20

Relative to static, the rolling horizon changes the mean collection by −0.4%, +0.4% and +0.7% and the distance by −6.0%, −8.8% and −8.1% in the low, base and high scenarios. Compared directly with the myopic policy, the mean difference (rolling minus myopic) is −0.76, +8.68 and +7.44 million VND, with $|t| \leq 1.4$ in all three cases. Figure 3 summarizes the comparison.

4.7 Discussion

Small instances. Up to 30 customers, HiGHS proves optimality in 10/10 runs in both phases, and the three metaheuristics find the same optimal value in every run. The one-day problem is solved at the root node, whereas the five-day problem with 30 customers needs on average 56.5 nodes and 19.6 s. With the same number of customers, linking the days through the schedule variables makes the problem clearly harder.

Behavior as size grows. The model size grows roughly as $|K|\,H\,n^2$ because of the arc variables, from 2,012 variables ($n = 30$, one day) to 67,610 ($n = 80$, five days). From $n = 50$ on, HiGHS reaches its time limit after at most 3.5 nodes on average in both phases: almost all the time is spent at the root node, and the incumbent is poor (gaps of 271% to 16,594% on INS_S), with a large variation between runs. The bound itself is informative, since the heuristics lie within 0.1–3.7% of it. These gaps therefore mostly reflect the solver's weak incumbent, not a weak bound.

Quality versus runtime. The metaheuristics reach gaps of 2.2–3.7% in Phase 1 and 0.4–2.6% in Phase 2 at $n = 80$, in 2 to 40 s. In Phase 2, ALNS gives the smallest gap on every instance

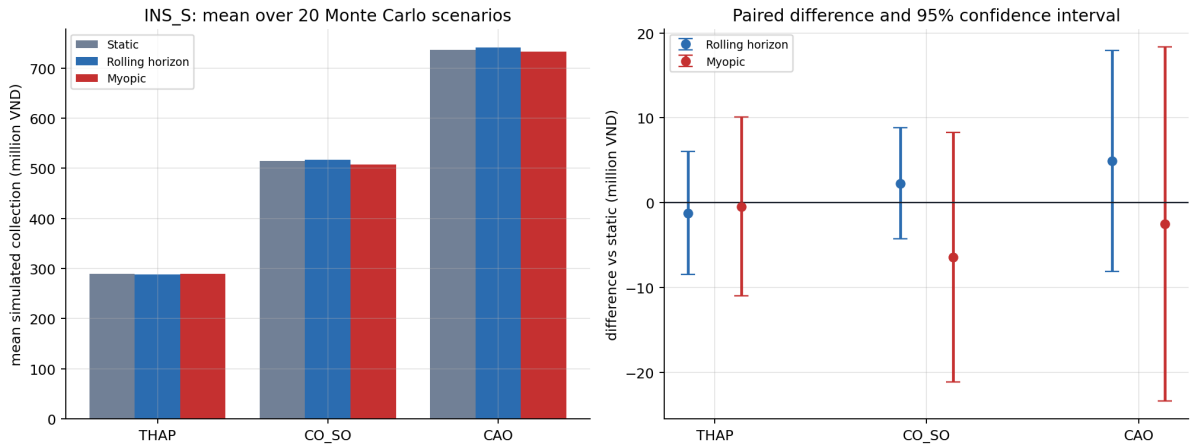


Figure 3: Policy comparison on INS_S. Left: mean simulated collection over 20 common payment scenarios (THAP = low, CO_SO = base, CAO = high). Right: paired difference against the static policy with its 95% confidence interval.

with $n \geq 50$ (0.08–0.49%) and the smallest variation between runs; Tabu is the fastest (5–7 s) with a gap of about 1%; GA is slower than Tabu, and at $n = 80$ it has the largest gap (1.6–2.6%) and the largest variation. In Phase 1 the three methods are within about 0.5 percentage points of each other, and none is best on all instances. The conclusions are the same on INS_S and on the independent REP sets (Table 5). Since the heuristics stop by an iteration rule while HiGHS runs to its time limit, the comparison is one of quality at the observed runtime, not at equal time.

Warm start. Starting HiGHS from the ALNS solution gives exactly the ALNS value and gap: within 150 s HiGHS neither improves the start nor closes the gap. On the 30-customer instance, the warm start reduces the solver time from 19.6 s to 1.9 s, because the optimal solution is provided from the start.

Benefits and limits of re-optimization. Re-planning every day does not change the simulated collection significantly on INS_S: all confidence intervals for the collection difference contain 0, and the rolling horizon is better than static in only 10–12 of the 20 scenarios. Its advantage does grow with the payment probability (−0.4%, +0.4%, +0.7%), which is expected: more payments mean more freed capacity and more information. The clear benefit is travel: the rolling horizon collects about the same amount with 6–9% less distance in every scenario group, because it drops planned repeat visits to customers who have already paid. It is also faster per scenario, since the remaining problems shrink. Two factors limit the gain. First, each re-solve optimizes an expectation as if no further re-planning would happen, so the policy is not optimal for the multi-stage problem. Second, ALNS is a heuristic, and the quality of consecutive solves varies by an amount comparable to the re-planning gain; in addition, the static plan received the full 300 ALNS iterations while each daily re-solve stopped early.

Variability. The collected money varies far more between payment scenarios (standard deviation 150–210 million VND) than between policies, which is why paired differences on common random numbers are needed. The myopic policy travels least, because it never plans repeat visits for later days, but its paired difference has a standard deviation 1.5–2.2 times that of the rolling horizon (22–45 against 14–28 million VND): it sometimes wins by a large margin and sometimes loses by one. With only 20 scenarios, none of the three policies can be called superior in collected money.

Figure 4 shows the five-day ALNS plan for INS_S in the base scenario. All 80 customers are visited, routes span most of the sampling area, and officer 1 visits only two customers on day 5.

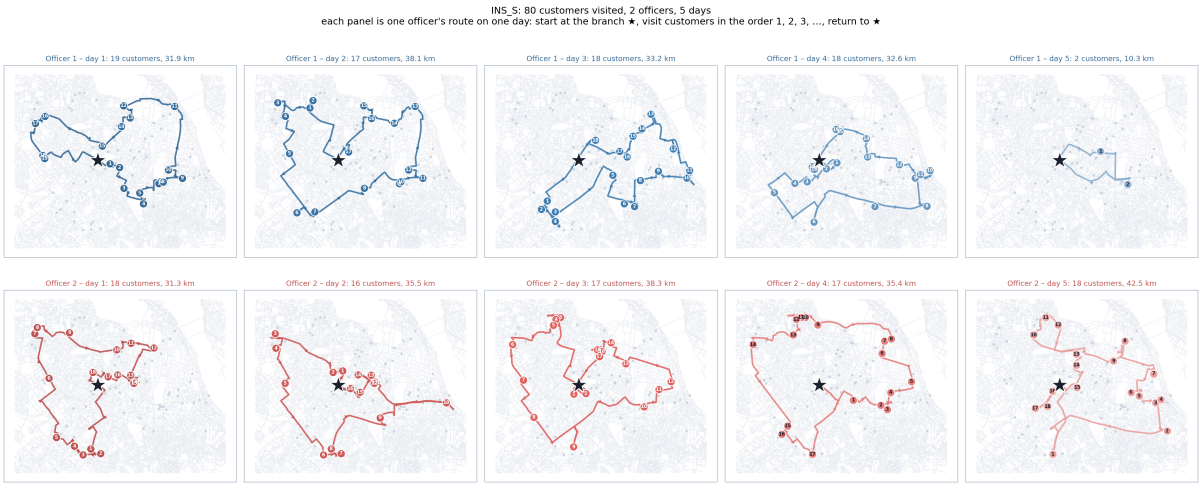


Figure 4: Five-day ALNS plan for INS_S (80 customers, 2 officers, base scenario). Each panel is one officer's route on one day; customers are numbered in visiting order, starting from and returning to the branch ($\star$).

5 Conclusion

This project studied a multi-officer, multi-day debt collection routing problem on the road network of Hanoi, with the goal of maximizing expected recovered money under daily working-time limits and a maximum number of visits per customer. The nonlinear value of repeated visits was linearized by selecting one precomputed visit schedule per customer, which gives a MILP with arc, assignment, route-activation, schedule and MTZ variables, plus symmetry breaking and an optional lexicographic distance objective. The model was solved with HiGHS and compared with a genetic algorithm, tabu search and ALNS, and a static plan was compared with rolling-horizon and myopic re-planning on common simulated payment scenarios.

The measured results suggest the following use of the methods. Up to 30 customers, HiGHS proves optimality within seconds and should be preferred, since it provides a certificate. From 50 customers on, HiGHS does not leave the root node within 30–150 s and its incumbent is unusable, although its bound remains useful to assess the heuristics. At this size the metaheuristics are the practical choice: ALNS gives the best plans for the five-day problem (gap 0.08–0.49%), and Tabu gives slightly weaker plans several times faster. Warm-starting HiGHS from ALNS keeps the ALNS quality but did not improve it within the time limit. In the policy comparison on 80 customers, daily re-planning did not change the simulated collection significantly, but it reduced travel distance by 6–9%.

These findings are limited by the data. Customer accounts are synthetic, the payment probabilities and their decline over time are modelling assumptions not calibrated on real collection outcomes, travel speeds ignore congestion, and payments are full or nothing. The size of the re-planning gain in particular depends on the assumed probabilities. The experiments also cover at most 80 customers and two officers, and the policy comparison uses only 20 payment scenarios.

Two directions are realistic next steps. First, running the policy comparison with more scenarios (the dataset provides 100) and on the larger instances INS_M and INS_L would show whether re-planning yields a significant gain in collection at larger scale. Second, the heuristic routes could be improved and the bounds tightened, for example by stronger subtour-elimination cuts than MTZ or a decomposition by day, so that exact methods remain usable beyond 30 customers.

References

[1] N. T. Phuc and D. T. L. Huong, *Mathematic Modelling Project: Source Code and Data*. Available at: `https://github.com/PhucNT2511/Mathematic-modeling-project`.

[2] OpenStreetMap contributors, *OpenStreetMap data for Vietnam* (Geofabrik extract `vietnam-latest.osm.pbf`, data up to 18 October 2026), Open Database License 1.0. Available at: `https://download.geofabrik.de/asia/vietnam.html`.

[3] NVIDIA, *Nemotron-Personas-Vietnam* [Dataset], CC BY 4.0, revision `950724d`, accessed 20 October 2026. Available at: `https://huggingface.co/datasets/nvidia/Nemotron-Personas-Vietnam`.

[4] I.-C. Yeh, *Default of Credit Card Clients* [Dataset], UCI Machine Learning Repository, 2009. DOI: `https://doi.org/10.24432/C55S3H`.

[5] C. E. Miller, A. W. Tucker, and R. A. Zemlin, "Integer programming formulation of traveling salesman problems," *Journal of the ACM*, vol. 7, no. 4, pp. 326–329, 1960.

[6] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.

[7] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Mathematical Programming Computation*, vol. 15, pp. 581–589, 2023.

[8] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.